Le jeu d'instructions assembleur 8086

Introduction

- Le jeu d'instructions permettant de programmer une machine en assembleur varie en fonction du processeur
- D'une manière générale les instructions d'un jeu peuvent toutefois être séparées en groupes:
  - Les instructions arithmétiques et logiques qui permettent d'effectuer des opérations
  - Les instructions de transfert permettant l'échange de contenu des variables ou des cases mémoire
  - Les instructions de test qui permettent de comparer des variables ou des cases mémoire, et modifie l'état des drapeaux
  - Les instructions de modification de séquence qui permettent de désigner la prochaine instruction à exécuter
- La description complète de chaque instruction est fournie par la documentation du processeur INTEL 8086

Les instructions de transfert

- Elles permettent de copier le contenu d'une variable, d'un registre ou d'une case mémoire dans une autre variable, registre ou case mémoire.
- Parmi ces instructions on trouve les instructions suivantes: MOV, XCHG, LEA, PUSH, POP, PUSHA, POPA

L'instruction MOV

- Elle prend deux opérandes:
  MOV dest , src

- Dest et src peuvent être:

```
MOV   reg,reg
MOV   mem, reg
MOV   reg, mem
MOV   mem, immediate data
MOV   reg, immediate data
MOV   ax/al, mem
MOV   mem, ax/al
MOV   segreg, mem16
MOV   segreg, reg16
MOV   mem16, segreg
MOV   reg16, segreg
```

L'instruction MOV

- Transfert de registre à registre:
  - `mov reg_destination, reg_source`

- Les deux registres doivent être de même taille

- Exemples:
  - `mov AX,BX`
  - `mov CL, AH`

L'instruction MOV

- Transfert d'un registre vers une cellule-mémoire (écriture en mémoire):
  - `mov mem, reg_source`

- L'information sera copiée en mémoire sur le nombre d'octets correspondant à la taille du registre

- La cellule mémoire peut être désigné par son adresse ou par un nom de variable associée

- Exemples:
  - `mov DS:[0002], DX`
  - `mov var1, AL  ; (en supposant que var1 a été déclarée dans le segment de données avec la directive DB)`
  - `mov [var1], AL`
  - `mov [var2], BX ; (en supposant que var2 a été déclarée dans le segment de données avec la directive DW)`
  - `mov var2, BX`

L'instruction MOV

- Chargement d'une valeur immédiate (constante) dans une une cellule-mémoire :
  - `mov mem, immediate_data`

- La cellule mémoire peut être désigné par son adresse ou par un nom de variable associée

- La valeur de la constante doit être en adéquation avec la taille de la zone mémoire

- Exemples:
  - `mov DS:[0002], 2`
  - `mov var1, 223  ; ( en supposant que var1 a été déclarée dans le segment de données avec la directive DB)`
  - `mov [var1], 223`
  - `mov [var2], 260 ; (en supposant que var2 a été déclarée dans le segment de données avec la directive DW)`
  - `mov var2, 260`

L'instruction MOV

- Chargement d'une valeur immédiate (constante) dans un registre:
  - `mov reg, immediate_data`
- La valeur de la constante doit être en adéquation avec la taille du registre
- Exemples:
  - `mov AL, 10011111b`
  - `mov CX, 223`
  - `mov AX,data ; une étiquette de segment est une constante`

L'instruction MOV

- Copie d'une valeur 16 bits en mémoire dans un registre segment:
  - `mov segreg, mem16`
- Les registres segments étant des registres 16 bits, la valeur lue en mémoire doit obligatoirement être une valeur sur 16 bits
- Le registre CS n'est jamais utilisé comme registre destination
- Exemples:
  - `mov ES,DS:[0004]`
  - `mov ES,var1 ;( en supposant que var1 a été déclarée dans le segment de données avec la directive DW)`

L'instruction MOV

- Copie d'une valeur contenue dans un registre général 16 bits dans un registre segment:
  - `mov segreg, reg16`
- Les registres segments étant des registres 16 bits, le registre source doit obligatoirement être un registre 16 bits
- Le registre CS n'est jamais utilisé comme registre destination
- Exemples:
  - `mov DS,AX`
  - `mov ES,SI`

L'instruction MOV

- Copie d'un registre segment vers une valeur 16 bits en mémoire (écriture):
  - `mov mem16, segreg`
- Les registres segments étant des registres 16 bits, la valeur écrite en mémoire est obligatoirement une valeur sur 16 bits
- L'emplacement mémoire peut être désigné par son adresse ou par un nom de variable associée
- Exemples:
  - `mov DS:[0004],ES`
  - `mov var1,ES ;( en supposant que var1 a été déclarée dans le segment de données avec la directive DW)`

L'instruction MOV

- Copie de la valeur d'un registre segment dans un registre général 16 bits:
  - `mov reg16,segreg`
- Les registres segments étant des registres 16 bits, le registre destination doit obligatoirement être un registre 16 bits
- Exemples:
  - `mov AX,DS`
  - `mov SI,ES`

L'instruction MOV

- Il n'est pas possible de copier le contenu d'une case mémoire dans une autre case mémoire
- Les opérandes sont soit deux registres, soit un registre et une cellule mémoire
- Il n'est pas possible d'effectuer un adressage immédiat dans un registre de segment, raison pour laquelle un programme assembleur nécessite les lignes suivantes d'initialisation du registre DS:

```
mov AX, data
mov DS, AX
```

où "data" est l'étiquette utilisée pour déclarer le segment de données

L'instruction MOV

- Les opérandes peuvent être de taille 8 ou 16 bits, mais ils doivent être de même taille

- Il peut être nécessaire de spécifier la taille de l'opération pour lever des ambiguïtés (cas de l'écriture de valeurs immédiates):

```
MOV    byte ptr [BX],0
MOV    word ptr [BX],0
```

- L'exécution d'une instruction MOV ne modifie pas les drapeaux d'état du microprocesseur

L'instruction XCHG

- L'instruction XCHG prend deux opérandes:

```
XCHG opérande1, opérande2
```

- Elle permet d'échanger le contenu des deux opérandes
- Les opérandes peuvent être de taille 8 ou 16 bits, mais ils doivent être de même taille
- Les opérandes peuvent désigner des registres ou des emplacements mémoire

L'instruction LEA

- LEA: Load Effective Address
- Permet de charger une adresse donnée dans un registre
- Syntaxe d'utilisation:

$$\begin{array}{ll} \text{LEA} & \text{dest, source} \\ \text{LEA} & \text{reg}_{16}\text{,mem} \end{array}$$

- Cette instruction stocke dans le registre **destinataire** l'adresse effective (partie *offset* de l'adresse logique) de la case mémoire spécifiée par **source**
- Le registre destination est nécessairement un registre 16 bits
- Exemples:
  - `lea SI, DS:[0006] ; (équivalent à mov SI,6)`
  - `lea SI, var1 ; (en supposant que var1 a été déclarée dans le segment de données)`

L'instruction LEA

- Exemples:

```
LEA   AX, DS :1234h
LEA   AX, [BX]
LEA   BX, [BX+3]
LEA   AX, [BX+3]
LEA   BX, [BP+SI+4]
```

Les instructions arithmétiques et logiques

- Le jeu d'instructions 8086 propose un grand nombre d'instructions arithmétiques et logiques
- Il s'agit d'opérations mathématiques (addition, soustraction, multiplication, division, …) et d'opérations booléennes (ET, OU, NOT, XOR, …) opérées sur des bits
- Les nombres manipulés par ces opérations sont exprimés en notation complément à 2
- Ces instructions peuvent changer les valeurs de certains drapeaux d'état du microprocesseur
- Le résultat est habituellement placé dans le premier opérande

Les instructions arithmétiques et logiques

- Quelques instructions:

ADD	Ajoute le deuxième opérande au premier.
SUB	Soustrait le deuxième opérande au premier.
CMP	Soustrait le deuxième opérande au premier pour les flags uniquement.
AND	Compare par un ET logique tous les bits des deux opérandes.
TEST	Idem à AND mais pour les flags uniquement.
OR	Effectue un OU logique entre tous les bits des deux opérandes.
XOR	Effectue un OU exclusif logique entre tous les bits des deux opérandes.

- Les deux opérandes de ces instructions peuvent être:

REG	mémoire
mémoire	REG
REG	REG
mémoire	immédiat
REG	immédiat

Les instructions arithmétiques et logiques

- Les registres utilisables par les opérations arithmétiques et logiques sont:

REG = AX, BX, CX, DX, AH, AL, BL, BH, CH, CL, DH, DL, DI, SI, BP, SP.

- Exemples:
  - *mise à zero du bit 3 de AX mais laisser le reste intact* *AND AL, F7h*
  - *mise à un du bit 3 de AX mais laisser le reste intact* *OR AL, 08h*

Les opérations logiques

- Le 8086 permet d'effectuer des opérations logiques grâce aux mnémoniques AND, OR, NOT et XOR.
- Ces opérateurs peuvent être utilisés avec :
  - AND/OR/XOR registre ou variable, registre
  - AND/OR/XOR registre, registre ou variable
  - AND/OR/XOR registre ou variable, constante
  - AND/OR/XOR registre ou variable, nombre
  - NOT registre ou variable

Les opérations logiques

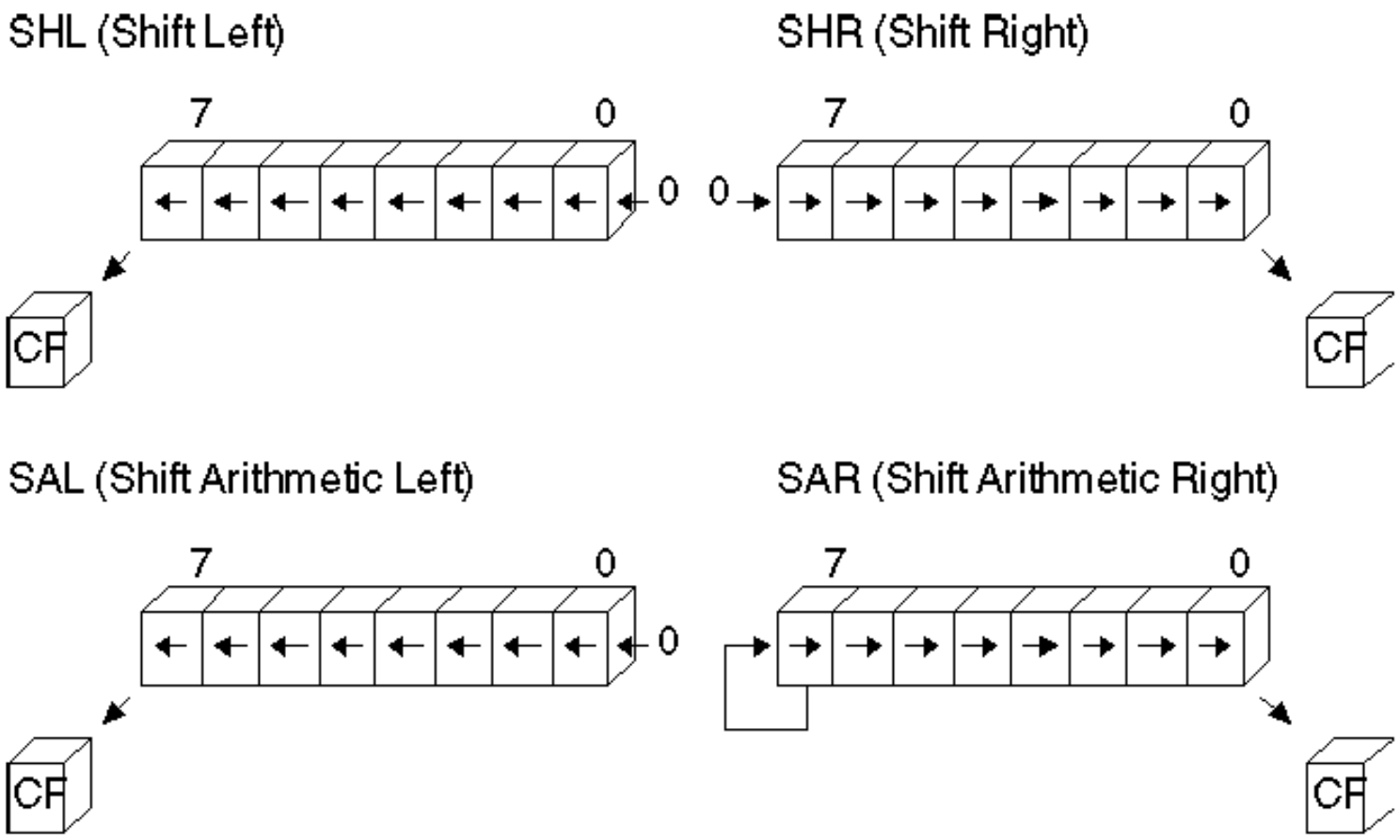
- Exemples:

```
MOV AL, 01100001b
AND AL, 11011111b ; --> donne 01000001b
OR AL, 01110110b ; --> donne 01110111b
NOT AL ; --> donne 10001000b
XOR AL,            01010101b ; --> donne
                   11011101b
XOR BX, BX ; --> donne 0000000000000000b
XOR AL, AL ; --> donne 00000000b
```

Les opérations de décalage

- Le 8086 permet d'effectuer des opérations de décalage sur des nombres signés ou non signés:
  - SHL (*SHift Left*) et SAL (*Shift Arithmetic Left*)
  - SHR (*SHift Right*) et SAR (*Shift Arithmetic Right*)
- Le nombre de bits de décalage est indiqué dans la seconde opérande
- Avec SHL, SAL et SHR, les bits qui sont expulsés sont stockés dans le bit CF
- Avec SAR le bit de signe (MSB) n'est pas concerné par le décalage, mais est conservé
- Les opérations de décalage sont plus rapides à exécuter que l'opération de multiplication ou de division par 2

Les opérations de décalage



Source illustration : http://www.sxlist.com/techref/language/masm/masmc04.htm

Les opérations de décalage

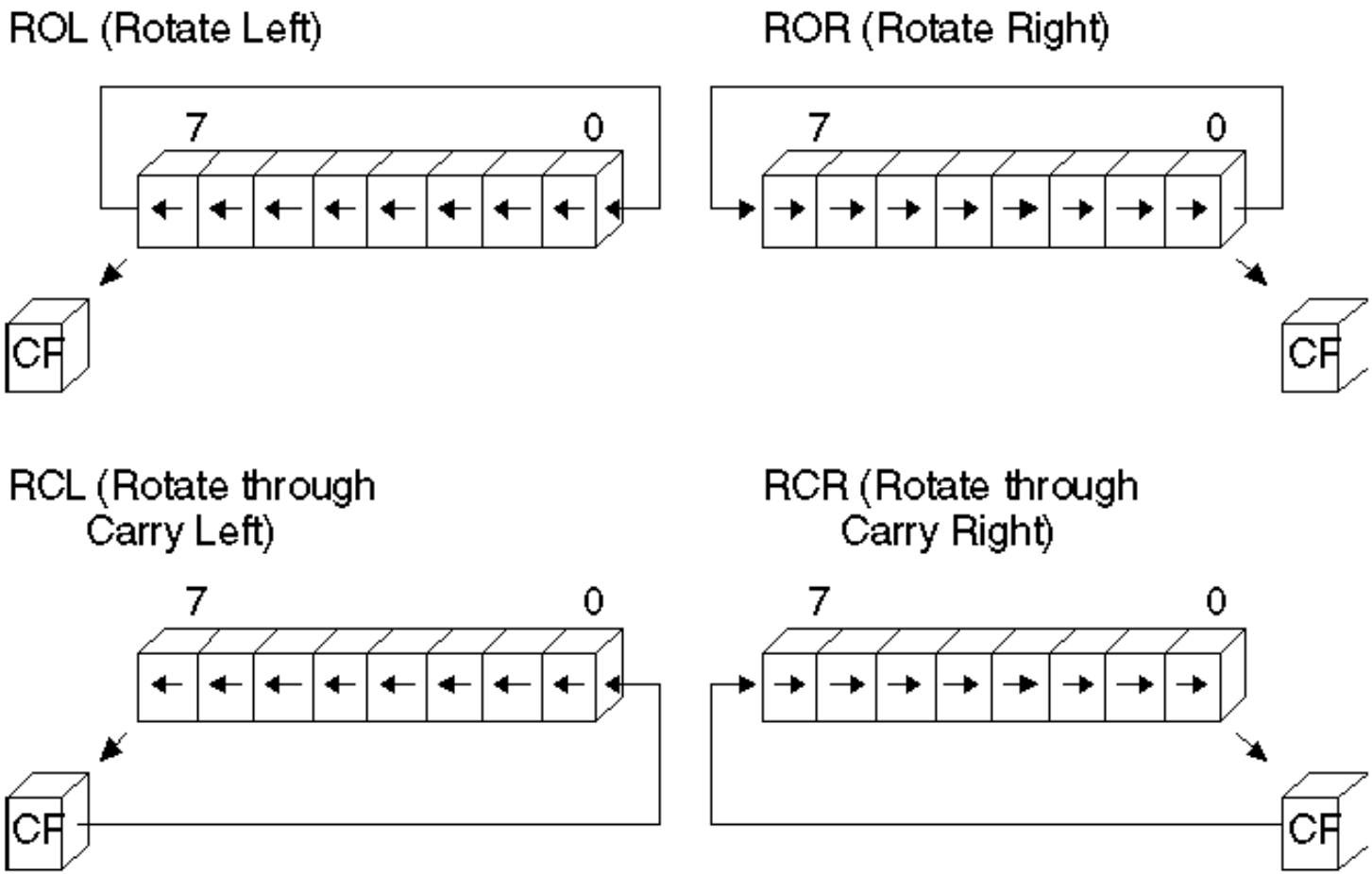
- Exemples:

```
MOV AL, 11110000b
SHL AL, 1 ; --> donne 11100000b
SAL AL, 1 ; --> donne 11000000b
SAR AL, 1 ; --> donne 11100000b
SHR AL, 1 ; --> donne 01110000b
```

Les opérations de rotation

- Le 8086 permet de réaliser des opérations de rotation sur des nombres signés ou non signés:
  - ROL (*Rotate Left*) et RCL (*Rotate Throw Carry Left*)
  - ROR (*Rotate Right*) et RCR (*Rotate Throw Carry Right*)
- Le nombre de bits de décalage est indiqué dans la seconde opérande
- Les bits expulsés sont replacés dans les trous formés par ce décalage
- Avec RCL et RCR le bit CF est utilisé en plus des bits à modifier (lors d'un mouvement, le trou est rempli avec une copie du bit CF et le bit expulsé est placé dans CF)

Les opérations de rotation



Source illustration : http://www.sxlist.com/techref/language/masm/masmc04.htm

Les opérations de rotation

- Exemples:

```
MOV AL, 11110000b
ROL AL, 1 ; --> donne 11100001b en supposant que CF=0
RCL AL, 1 ; --> donne 11000010b et CF = 1
RCR AL, 1 ; --> donne 11100001b et CF = 0
ROR AL, 1 ; --> donne 01111000b
```

Les opérations arithmétiques

- Les mnémoniques ADD (***ADD**ition*) et SUB (***SUB**traction*) permettent d'effectuer des additions et des soustractions:
  - ADD registre, registre ou variable
  - ADD registre ou variable, registre
  - ADD registre ou variable, constante
  - SUB registre, registre ou variable
  - SUB registre ou variable, registre
  - SUB registre ou variable, constante
- Si une addition ou une soustraction sont codées sur 16 bits, alors leurs résultats seront codés sur au plus 16 + 1 bits soit 16 bits de résultat + 1 bit de retenue (le bit CF)
- Les opérations ADD et SUB sont effectuées sans tenir compte de l'état initial du bit CF du registre d'état mais l'état de ce bit peut être modifié à l'issue de l'opération

Les opérations arithmétiques

- L'instruction ADC permet d'effectuer une addition qui tient compte du bit CF
- L'instruction ADC modifie l'état du bit CF et donc sert à effectuer des opérations sur 48 bits, 64 bits …
- En stockant les résultats intermédiaires en mémoire, l'opération se fera par bloc de 16 bits.
- Idem pour la soustraction avec l'instruction SBB

Les opérations arithmétiques

- L'instruction CMP permet de comparer deux valeurs données en opérandes (registres ou variables)
- En pratique elle soustrait la source de la destination
- Elle ne stocke pas le résultat de cette soustraction dans la destination
- Elle positionne les indicateurs du registre d'état afin qu'ils puissent être exploités ensuite par des instructions de branchement conditionnel

	opérandes non signés				**opérandes signés**			
	OF	SF	ZF	CF	OF	SF	ZF	CF
source < destination	-	-	0	0	0/1	0	0	-
source = destination	-	-	1	0	0	0	1	-
source > destination	-	-	0	1	0/1	1	0	-

Les opérations d'incrémentation et décrémentation

- Le 8086 propose 2 instructions INC et DEC pour incrémenter ou de décrémenter :
  - INC registre ou variable
  - DEC registre ou variable
- INC AX incrémente la valeur stockée dans AX. INC AX donne le même résultat que ADD AX, 1 mais le premier mnémonique correspond à un codeop de 1 octet alors que le codeop correspondant au second mnémonique en occupe 3.
- DEC BX décrémente la valeur stockée dans BX. DEC BX donne le même résultat que SUB BX, 1 mais le premier mnémonique correspond à un codeop de 1 octet alors que le codeop correspondant au second mnémonique en occupe 3.

Les opérations de multiplication et division

- Les instructions de multiplication et division MUL, IMUL, DIV, IDIV ne prennent qu'un seul opérande (registre ou mémoire)
- Les registres utilisables sont: AX, BX, CX, DX, AH, AL, BH, BL, CH, CL, DH, DL, DI, SI, BP et SP
- Les instructions MUL et IMUL affectent uniquement les drapeaux CF et OF (1 lorsque le résultat est plus grand que la taille de l'opérande, sinon 0)
- Pour DIV et IDIV les drapeaux ne sont pas définis

Les opérations de multiplication et division

MUL	Multiplication non signée	byte : AX = AL * opérande word : (DX AX) = AX * opérande.
IMUL	Multiplication signée	byte : AX = AL opérande word : (DX AX) = AX * opérande.
DIV	Division non signée	byte : AL = AX / opérande ; AH = reste (modulo). word : AX = (DX AX) /opérande ; DX = reste (modulo)...
IDIV	Division signée	byte : AL = AX /opérande ; AH = reste (modulo) word : AX = (DX AX) /opérande ; DX = reste (modulo)

Les instructions de branchement

- Les instructions sont exécutées de manière séquentielle
- Le cycle d'exécution d'une instruction modifie IP de manière à ce que ce registre pointe sur la prochaine instruction à exécuter
- L'instruction en cours d'exécution peut être amenée à modifier ce compteur
- Les instructions qui permettent cela sont les instructions de branchement
- Ces instructions permettent d'exécuter des instructions de manière non séquentielle

Les instructions de branchement

- Très utiles car elles permettent de faire répéter une suite d'instructions au processeur ou de déclencher une action en fonction du résultat d'une opération
- Les instructions de branchement modifient le séquencement des instructions à exécuter en modifiant le contenu du registre IP
- 3 types d'instructions de branchement:
  - Sauts inconditionnels
  - Sauts conditionnels
  - Appels de sous-programmes

Les sauts inconditionnels

- L'instruction JMP permet d'exécuter une instruction qui se trouve à n'importe quel endroit du programme
- L'instruction à exécuter est usuellement référencée par une étiquette ou une adresse
- Le fait d'utiliser une étiquette permet de faire un référencement implicite (référencement relatif) et d'être insensible à des rajouts de ligne de code dans le programme

Les sauts inconditionnels

- Exemple:

```
debut :   MOV   AX, 0
boucle :  MOV   [0140h], AX
          JMP   boucle
          MOV   AX,1
```

Adresse	Contenu mém	Langage Mnemonique	Explication en francais
0100h	B8 00 00	MOV AX, 0	charge zero dans AX
0103	A3 40 01	MOV [0140h], AX	écrit le contenu de AX à l'adresse 0140h
0106	EB FB	JMP 103	Branche à l'adresse 0103
0107	B8 01 00	MOV AX,1	charge un dans AX

Les sauts conditionnels

- Ce sont des instructions permettant de créer des boucles conditionnées par une valeur booléenne
- Ces instructions testent un ou plusieurs drapeaux du registre d'état, et effectue le branchement si le test est vérifié
- Si le test n'est pas vérifié le processeur exécute normalement la prochaine instruction dans le programme

Les sauts conditionnels

- Les principales instructions de saut conditionnel:

Instruction	Description	Condition	Aliases	Opposite
JC	Jump if carry	CF = 1	JB, JNAE	JNC
JNC	Jump if no carry	CF = 0	JNB, JAE	JC
JZ	Jump if zero	ZF = 1	JE	JNZ
JNZ	Jump if not zero	ZF = 0	JNE	JZ
JS	Jump if sign	SF = 1		JNS
JO	Jump if overflow	OF = 1		JNO
JNO	Jump if no overflow	OF = 0		JO
JP	Jump if parity	PF = 1	JPE	JNP
JPE	Jump if parity even	PF = 1	JP	JPO
JNP	Jump if no parity	PF = 0	JPO	JP
JPO	Jump if parity odd	PF = 0	JNP	JPE

Les sauts conditionnels

- Pour des comparaisons sur des nombres non signés les instructions suivantes sont utilisables:

Instruction	Description	Condition	Aliases	Opposite
JA	Jump if above (>)	CF = 0 and ZF = 0	JNBE	JNA
JAE	Jump if above or equal (≥)	CF = 0	JNC, JNB	JNAE
JB	Jump if below (<)	CF = 1	JC, JNAE	JAE
JBE	Jump if below or equal (≤)	CF = 1 or ZF = 1	JNA	JA
JE	Jump if equal to (=)	ZF = 1	JZ	JNE
JNE	Jump if not equal to (≠)	ZF = 0	JNZ	JE

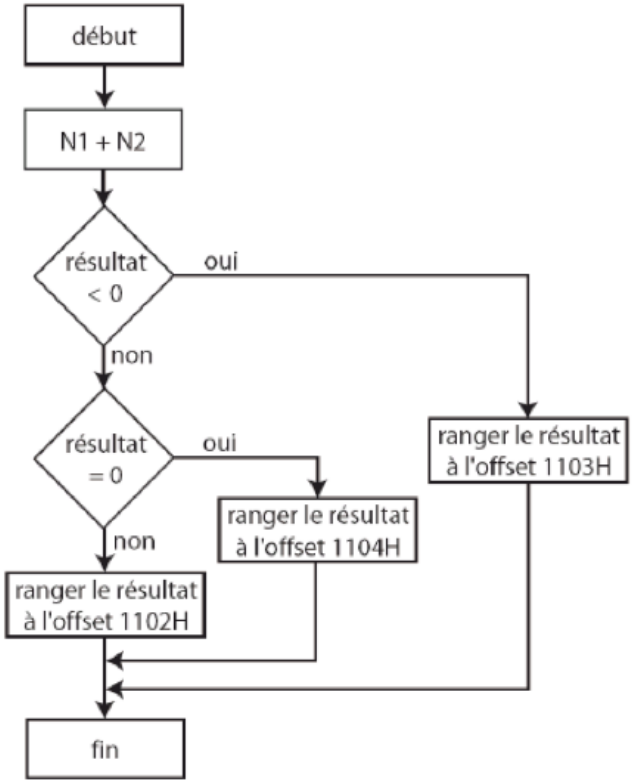
Les sauts conditionnels

- Pour des comparaisons sur des nombres signés les instructions suivantes sont utilisables:

Instruction	Description	Condition	Aliases	Opposite
JG	Jump if greater ($>$)	SF $=$ OF and ZF $= 0$	JNLE	JNG
JNLE	Jump if not less than or equal to (not $\leq$)	SF $=$ OF and ZF $= 0$	JG	JLE
JGE	Jump if greater than or equal ($\geq$)	SF $=$ OF	JNL	JNGE
JNL	Jump if not less than (not $<$)	SF $=$ OF	JGE	JL
JL	Jump if less than ($<$)	SF $\neq$ OF	JNGE	JNL
JNGE	Jump if not greater than or equal to (not $\geq$)	SF $\neq$ OF	JL	JGE
JLE	Jump if less than or equal ($\leq$)	SF $\neq$ OF or ZF $= 1$	JNG	JNLE
JNG	Jump if not greater than (not $>$)	SF $\neq$ OF or ZF $= 1$	JLE	JG
JE	Jump if equal to ($=$)	ZF $= 1$	JZ	JNE
JNE	Jump if not equal to ($\neq$)	ZF $= 0$	JNZ	JE

Les sauts conditionnels

- Exemple de programme




```
                mov     al,[1100H]
                add     al,[1101H]
                js      negatif
                jz      nul
                mov     [1102H],al
                jmp     fin
negatif :       mov     [1103H],al
                jmp     fin
nul :           mov     [1104H],al
fin :           ; suite du programme
```

Les sauts conditionnels

- Exemple 2: calcul factorielle

```
Etiquette       Instruction                          Commentaires

DebutProg :     CMP CX,0                             ; test si CX vaut 0
                JNZ TestSup8                         ; test si CX vaut 0
                MOV AX,1                             ; CX contient 0, 0!=1
                JMP FinDuProg                        ; Saut à la fin du programme
TestSup8 :      CMP CX,8                             ;test si CX vaut plus que 8
                JNG Entre1et8                        ;Calcul le factoriel si CX n'est pas trop grand
                MOV AX,0FFFFh                        ;CX trop grand, mettons AX à 65365
                JMP FinDuProg                        ;Saut à la fin du programme
Entre1et8 :     MOV AX,1                             ;pour la première multiplication
                MOV BX,CX                            ;Initialisation de BX
CalculFacto :   IMUL BX                              ;AX=AX*BX
                DEC BX                               ;Decrementation de BX
                JNZ CalculFacto                      ;Arrete  si BX = 0
```